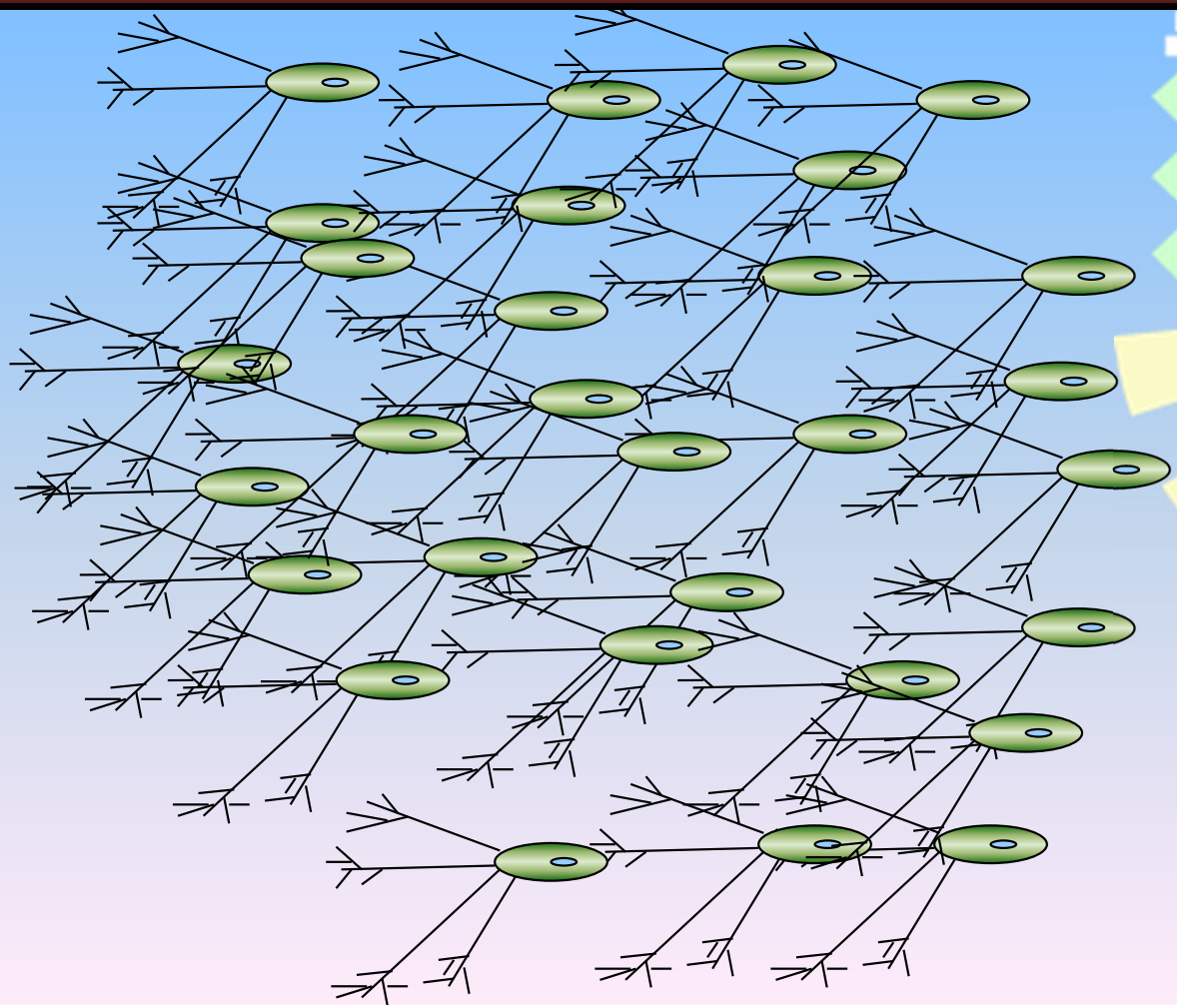
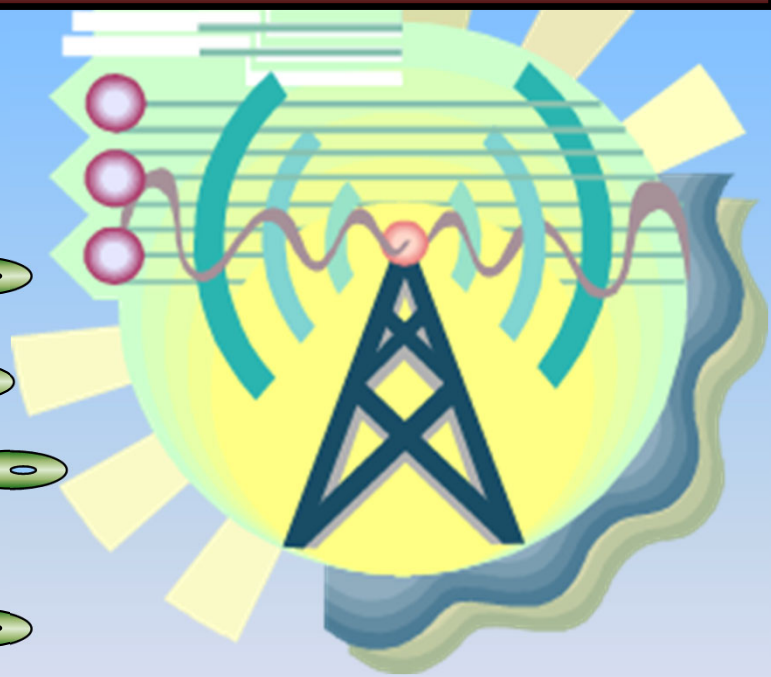
Neural Networks

Part 3





n.h.reyes@massey.ac.nz

Neural Networks

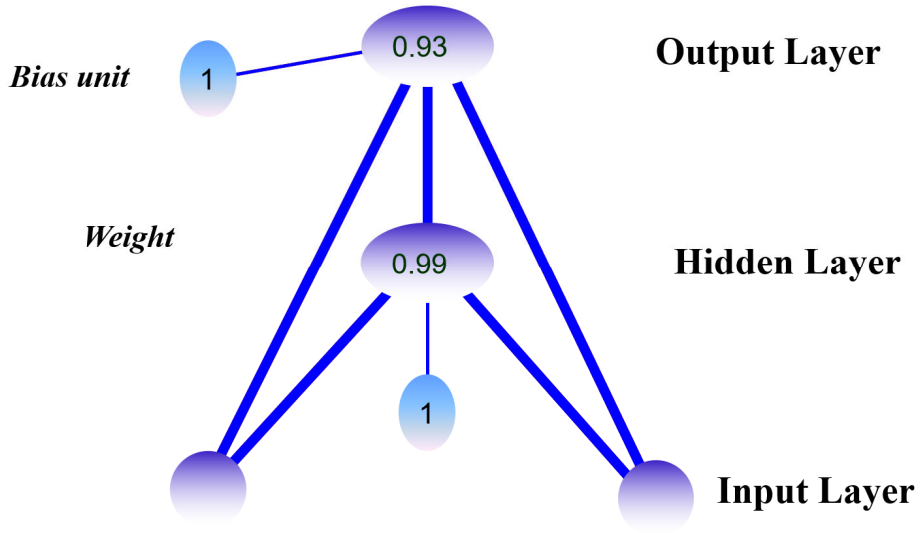
- [x] Introduction to Multi-layer Feed-forward Network
- [x] Evaluating a Feed-forward Network
- [x] Training a Feed-forward Network
- [x] Theory (Derivation of Formulas)
- [x] Review of Formulas, Variations in Weight Updates
- [ ] Overfitting Problem, Softmax units, Performance Metrics
- [ ] Variations in Activation Function
- [ ] Input/Output Representations, Bias Units, Hyperparameter, Data Shuffling
- [ ] Experiments using Tensorflow playground

Possible Training Problem: getting stuck at a local minimum

Training a Network

BACKPROPAGATION TRAINING

One unfortunate problem with backprop is that **when the learning rate is too large** the **training can fail** as it did in the case of the neural network we considered for solving the XOR problem, when $\eta = 3.0$. Here, after **10,000** epochs the results were:



x	y	$z_{desired}$	z_{actual}
1	0	1	0.994
0	0	0	0.009
0	1	1	0.994
1	1	0	1.000

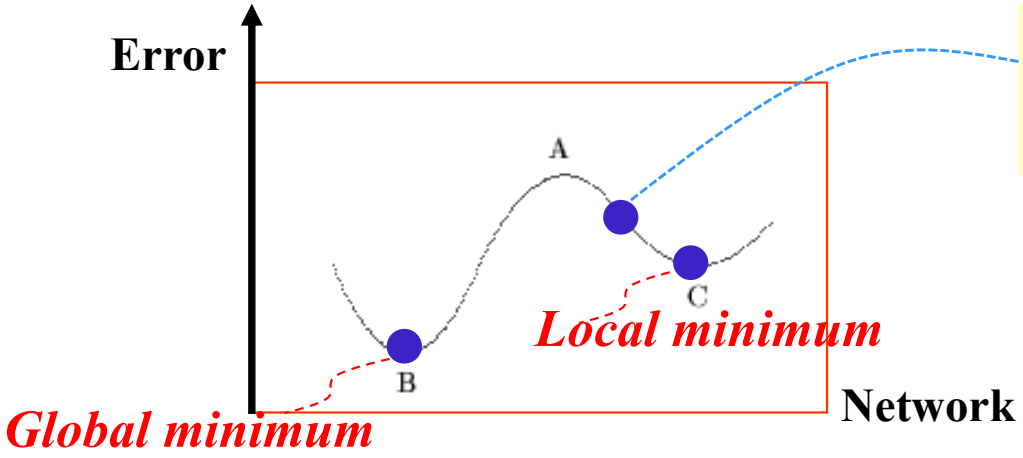
Incorrect result!

the output for the last pattern is **1** not **0**.

Training a Network

BACKPROPAGATION TRAINING

The geometric interpretation of this problem is that when the network tries to make the error go down the network may get stuck in a valley that is not the lowest possible valley.



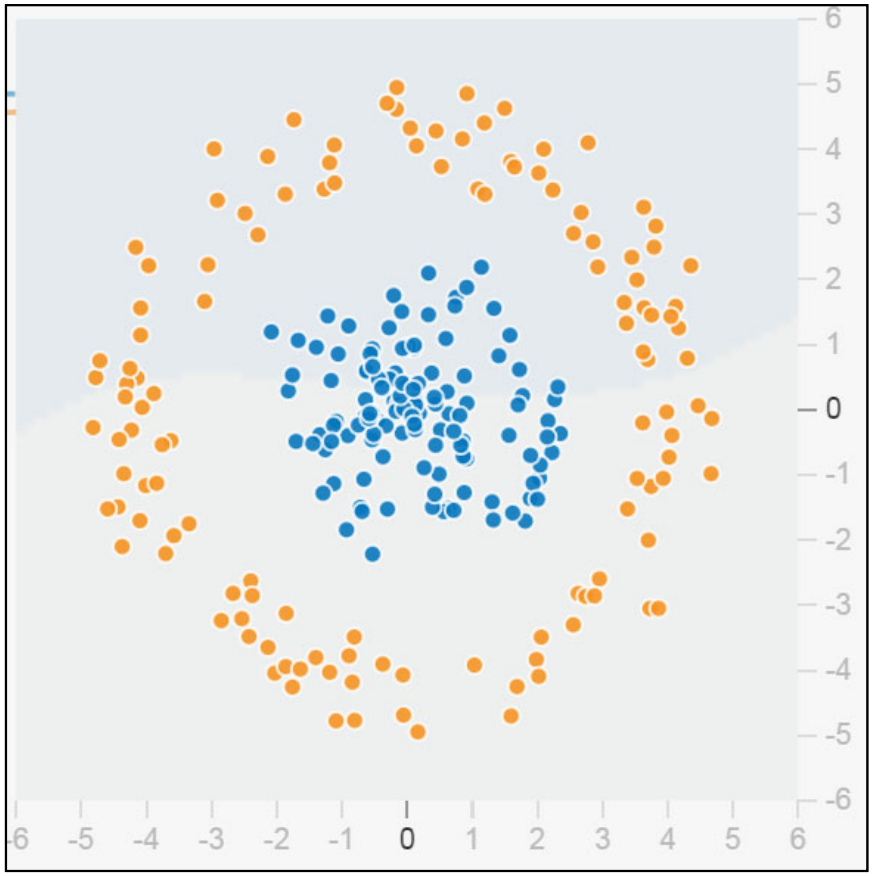
Each point in the curve corresponds to a complete neural network configuration

When backprop starts at point ***A*** and tries to minimize the error, you hope the process will stop when it hits the low point at ***B*** however you could get unlucky and hit the not so low point at *C*. The lowest point is a ***global minimum*** and the not so low point is a ***local minimum***.

Training a Network

Example:

Classification problem: distinguish the blue dots from the orange dots



Demonstration: **playground.tensorflow.org**

Training a Network

Example: Consequence of setting the learning rate too high.

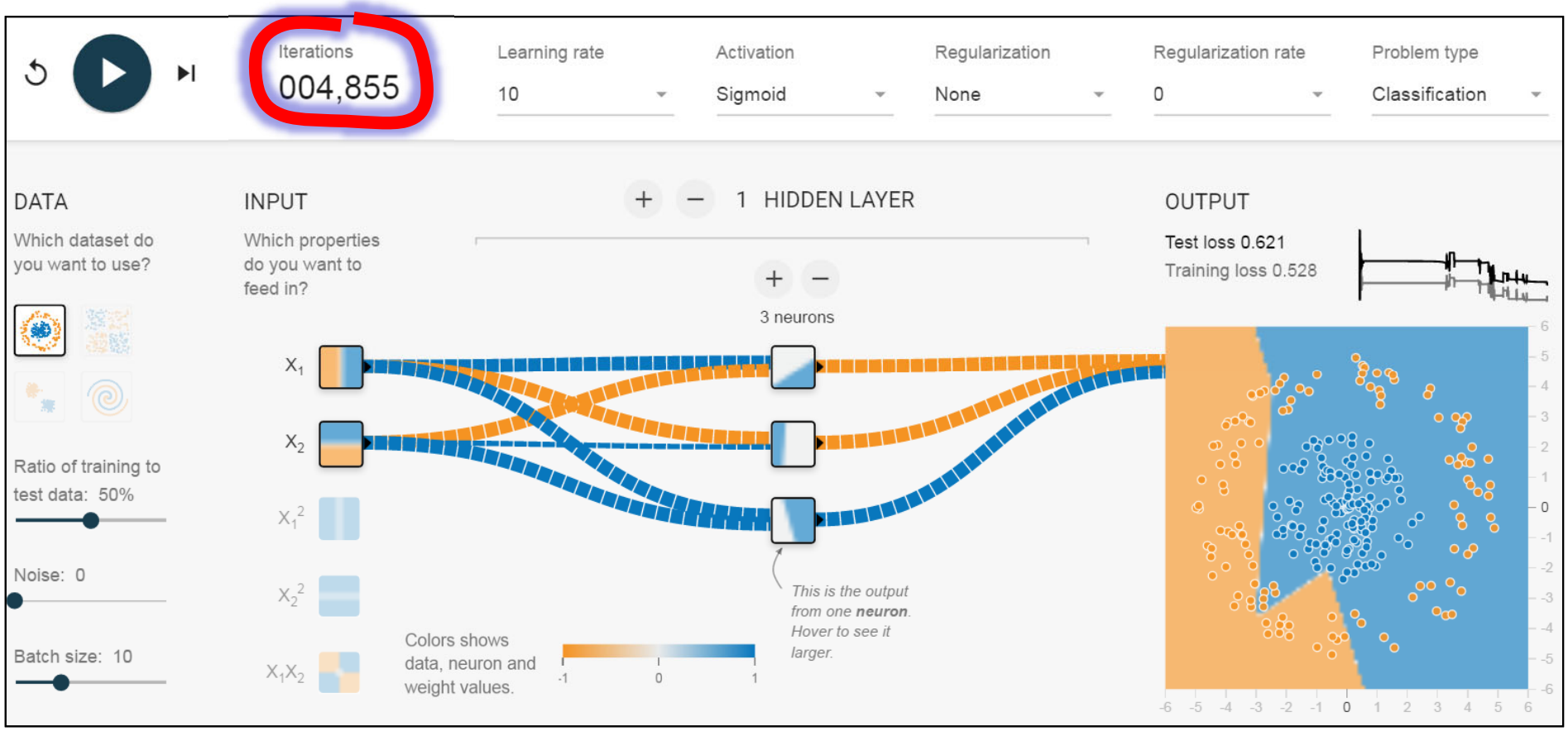


Regardless of how long you train the network, its performance does not significantly improve anymore.

playground.tensorflow.org

Training a Network

Example: Consequence of setting the learning rate too high.

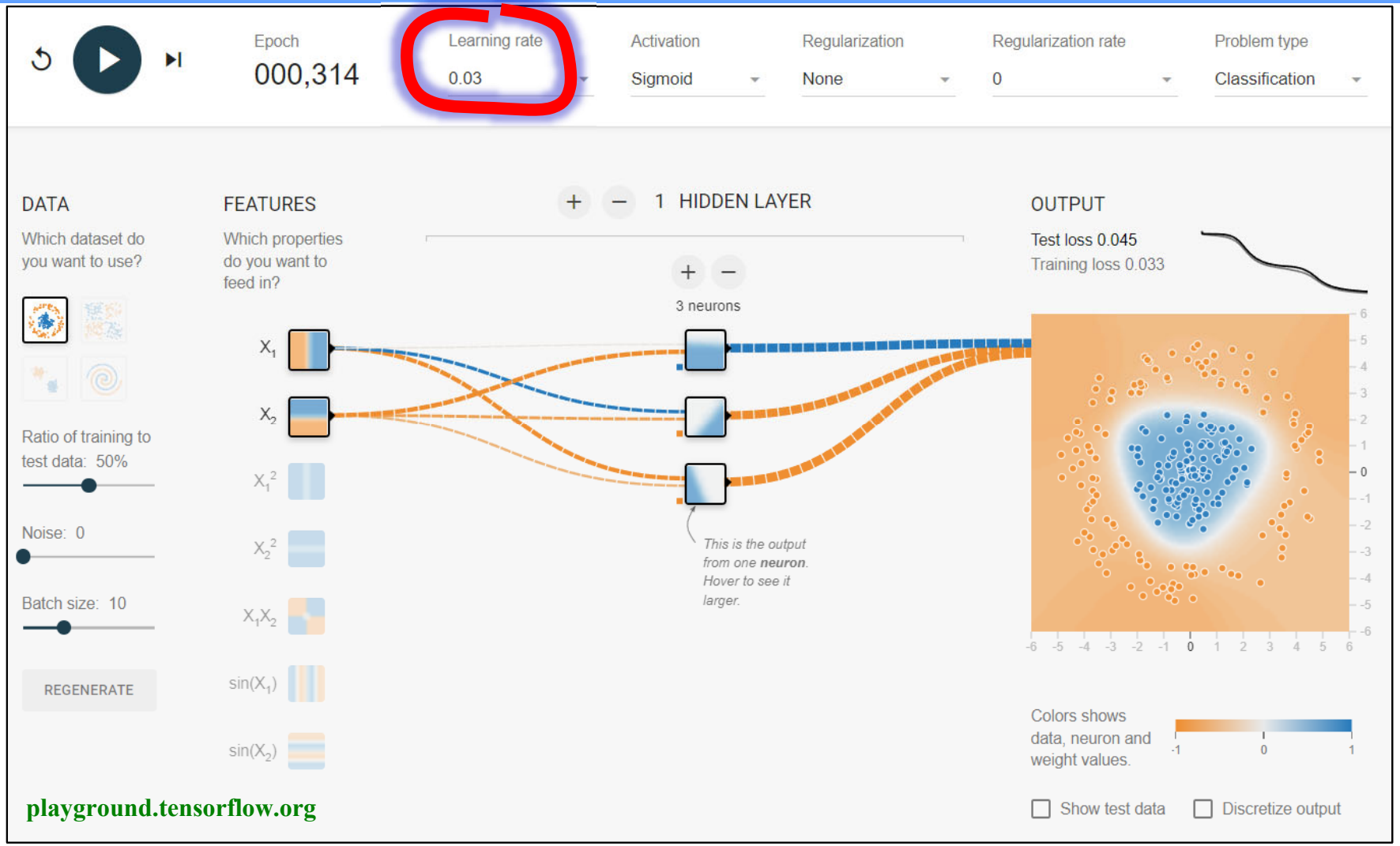


Regardless of how long you train the network, its performance does not significantly improve anymore.

playground.tensorflow.org

Training a Network

Example: Problem fixed!



After setting the learning rate to **0.03**, it can be observed that the network is now able to isolate the blue dots from the orange dots.

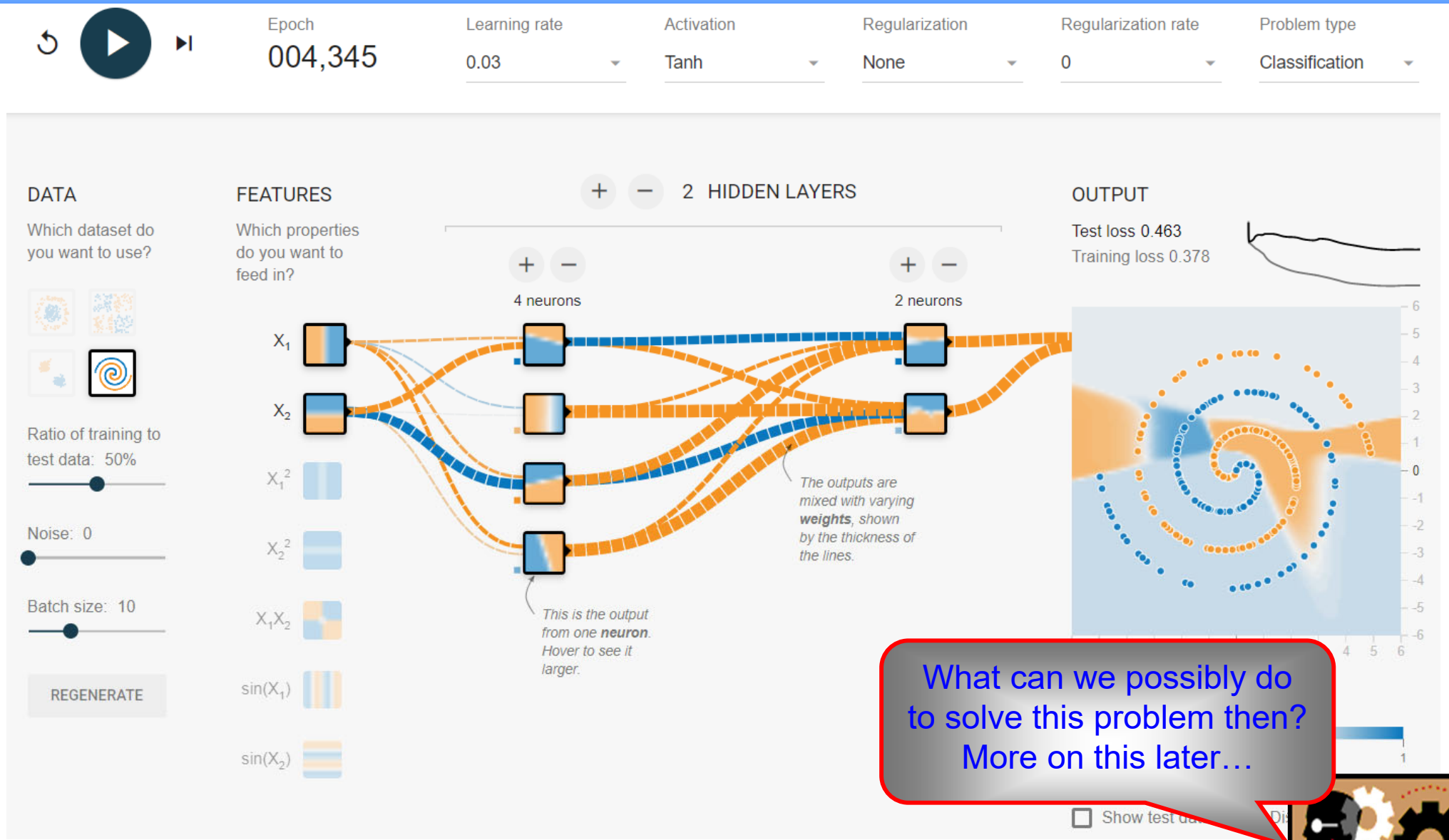
Spiral Data Set

More challenging classification problem



Training a Network

Example: Unable to solve problem!



Complete Training

- **Given:** Data set, desired outputs and a Neural Net with m weights.
- **Find a setting for the weights that will give good predictive performance on new data. Estimate expected performance on new data.**

1. **Split data set (randomly) into three subsets:**
   - **Training set – used for picking weights**
   - **Validation set – used to stop training**
   - **Test set – used to evaluate performance**
2. **Pick random, small weights as initial values.**
3. **Perform iterative minimization of error over training set.**
4. **Stop when error on validation set reaches a minimum (to avoid over-fitting).**
5. **Repeat training (from Step 2) several times (to avoid local minima).**
6. **Use best weights to compute error on test set, which is the estimate of performance on new data. Do not repeat training to improve this.**



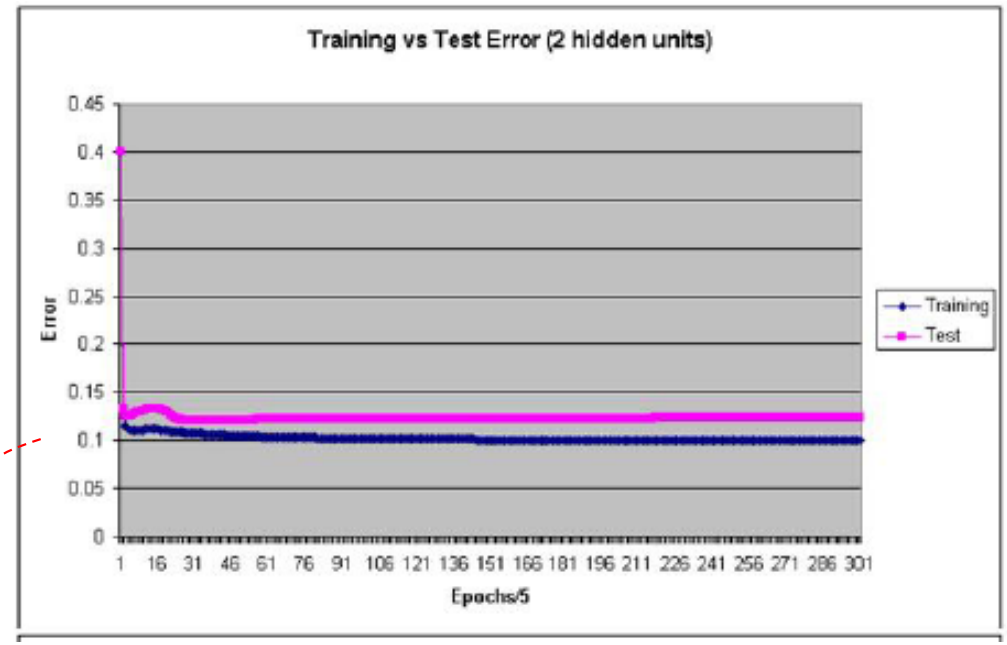
Notes on Training

1. pick a set of **random small weights** as the initial values of the weights. This reduces the chance of saturating any of the units initially.
2. we do not want to simply keep going until we reduce the training error to its minimum value. This is likely to **overfit** the **training data**.
3. stop when we get best the performance on the **validation set**
4. Keeping the weights small is a strategy for reducing the size of the hypothesis space. It also reduces the variance of the hypothesis since it limits the impact that any particular data point can have on the output.

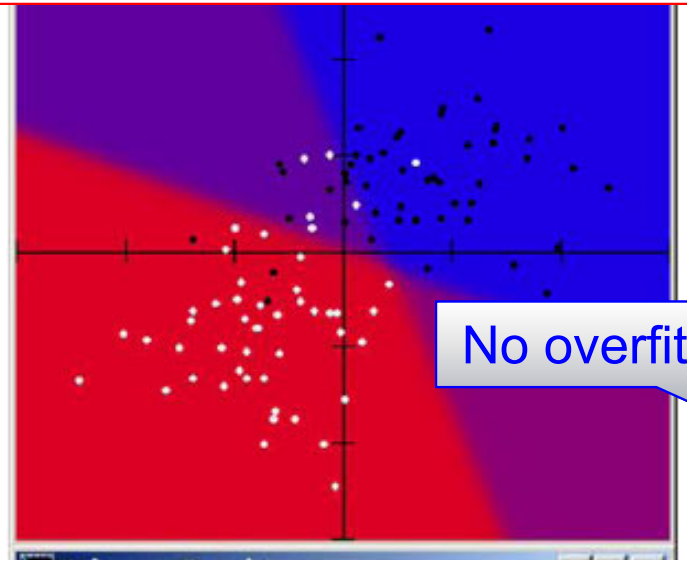


How can we tell if a network is learning well?

Training without a validation set



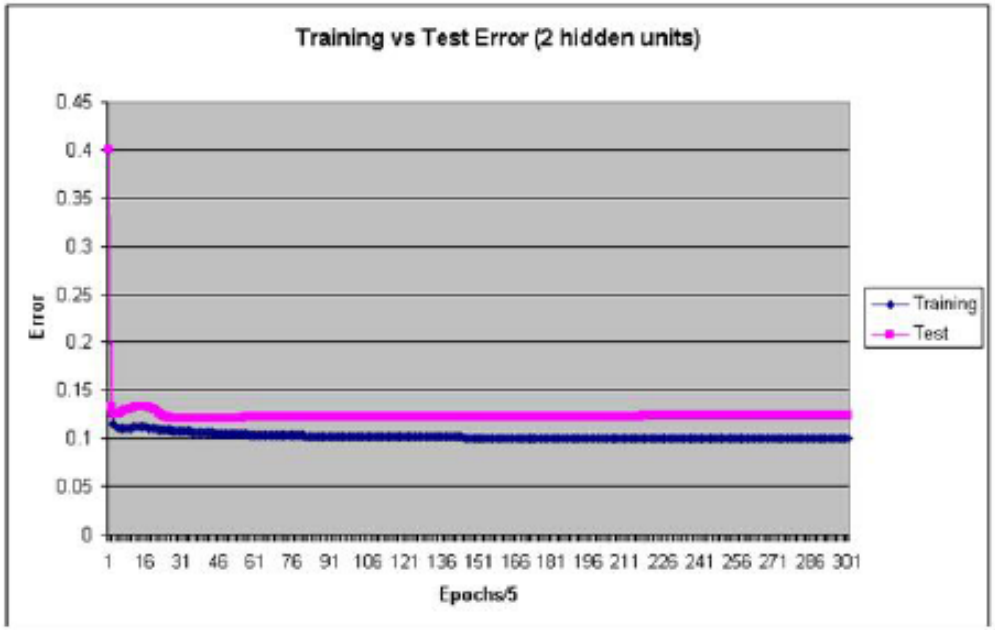
2 inputs, 2 hidden units, 1 output

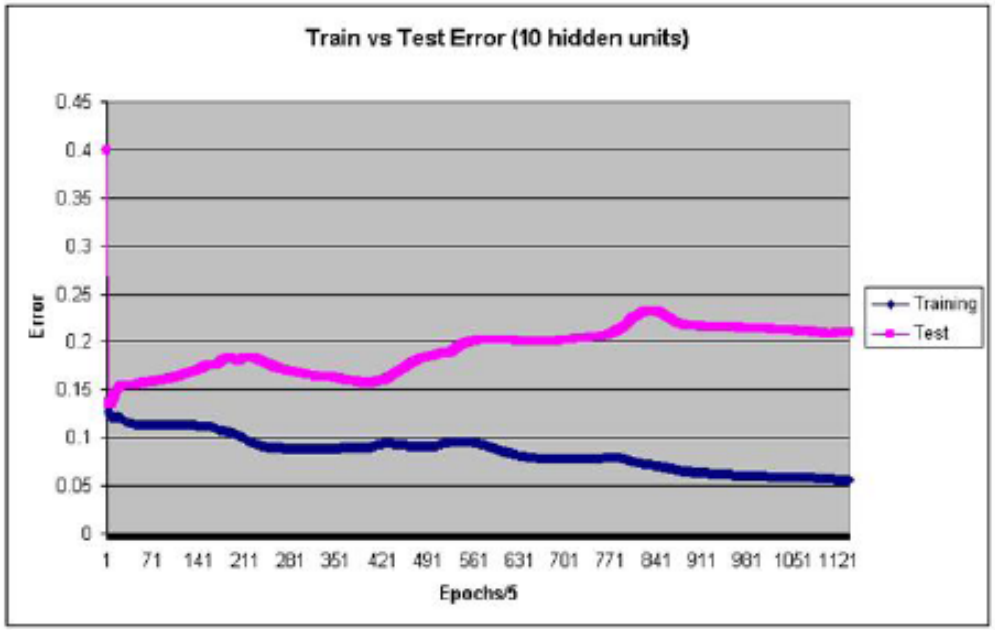


No overfitting!

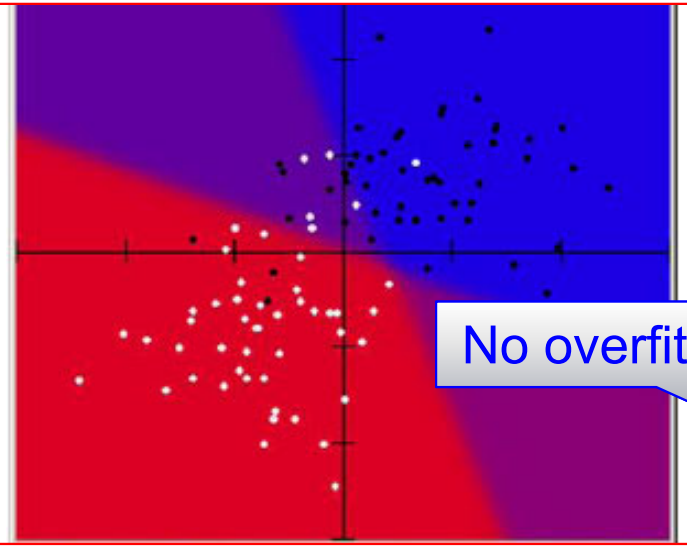
Examine the performance of the network on both the training set and test set.

Training without a validation set



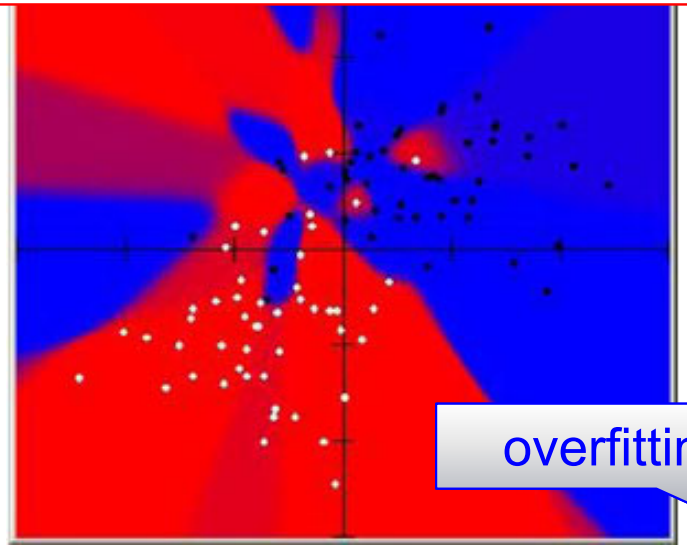


2 inputs, 2 hidden units, 1 output



No overfitting!

2 inputs, 10 hidden units, 1 output



overfitting!

Early Stopping

- Choose the stopping point (stop training before going too far astray) based on the network's performance on **validation data.**
- This technique is also used to prevent overfitting.



Ockham's Razor

- *entia non sunt multiplicanda praeter necessitatem*
- "entities must not be multiplied beyond necessity"
  - William of Ockham
  - (14th century Franciscan friar, logician, theologian)



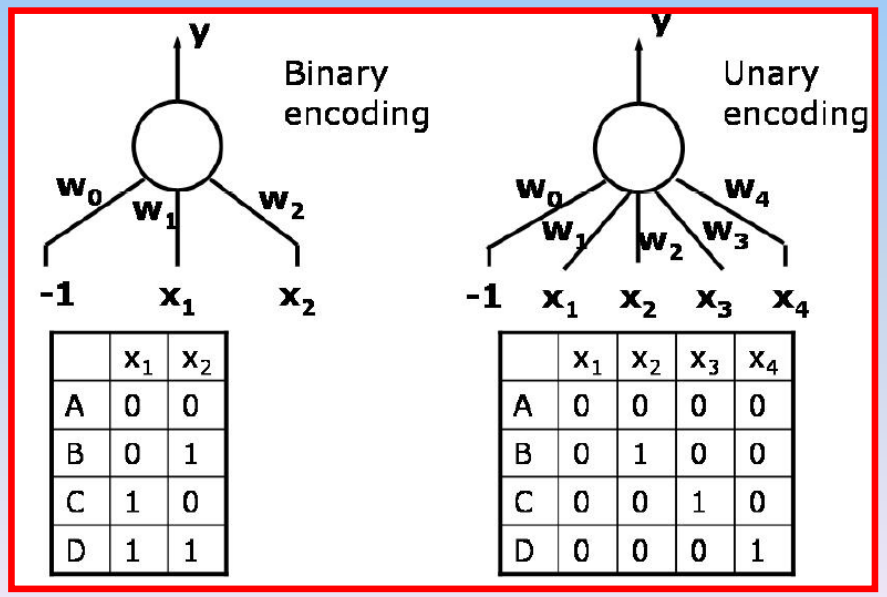
Input & Output Representations

Input Representation

- All the signals in a neural net are **[0, 1]**. Input values should also be **scaled** **to this range** (or approximately so) so as to speed up training.

Input Representation

- All the signals in a neural net are **[0, 1]**. Input values should also be **scaled to this range** (or approximately so) so as to speed up training.
- If the input values are **discrete**, **e.g. {A,B,C,D}** or **{a,g,t,c},** they need to be coded in unary form.



	x_1	x_2
A	0	0
B	0	1
C	1	0
D	1	1

	x_1	x_2	x_3	x_4
A	0	0	0	0
B	0	1	0	0
C	0	0	1	0
D	0	0	0	1

Output Representation

- A neural net with a single sigmoid output is aimed at binary classification.

  Class is **0** if **y** < **0.5** and **1** otherwise.

- **For multi-class problems**
  - Can use one output per class (unary encoding)
  - There may be confusing outputs (two outputs > 0.5 in unary encoding)
  - More sophisticated method is to use special ***softmax units***, which force outputs to sum to 1.

Softmax Units

- For multi-class problems, use softmax units
- Softmax activation function:

net input

$$f_q(k) = z_k^* = \frac{\exp(z_k)}{\sum_{k_1} \exp(z_{k_1})}$$

Actual output

e.g.

net input 1	net input 2	net input 3	net input 4	net input 5
0.776315846	0.528655919	0.498515491	0.613505773	0.339863291

Number of Classes: 5

denominator 8.768

Softmax Units (Outputs)

Output 1	Output 2	Output 3	Output 4	Output 5	SUM
0.247883624	0.193504245	0.187758962	0.210639745	0.160213424	1

Softmax Units

- When the inputs to the Softmax unit gets too high, it is possible for the output to become NAN
- The reason is that the computer cannot handle such big numbers; therefore, returning Not A Number result.
- One possible approach to solving this problem is to shift the highest value in the inputs to zero, then compute for the Softmax.

- For example, to compute the softmax of [1, 3, 5] use [1-5, 3-5, 5-5] which is [-4, -2, 0].

Softmax and Cross Entropy Pairing

Softmax activation function

$$f_q(k) = z_k^* = \frac{\exp(z_k)}{\sum_{k_1} \exp(z_{k_1})}$$

Cross-Entropy Penalty Function

$$\rho_c = \sum_{n=1}^{N} \sum_{k=1}^{q} t_{nk} \log\left(\frac{t_{nk}}{z_{nk}^*}\right).$$

Target output for output node k, for the given pattern n

Actual output of output node k, given pattern n

*Cross-entropy can be used as an error measure when a network's outputs can be thought of as representing independent hypotheses (e.g. each node stands for a different concept), and the node activations can be understood as representing the probability (or confidence) that each hypothesis might be true.

http://www.cse.unsw.edu.au/~billw/cs9444/crossentropy.html

*excerpt from p. 166 of Plunkett and Elman: Exercises in Rethinking Innateness, MIT Press, 1997

Error delta at the output node(s)

$$\delta_k = \overbrace{(t_k - O_k)}^{\frac{dP}{do}} \overbrace{Ok(1 - O_k)}^{\frac{dO_k}{dp_r}}$$

Derivative of the Sigmoid function with respect to its input

$O_k - activation\ function\ of\ output\ node\ k$
$p_r - weighted\ input\ of\ output\ node\ k$
$t_k - target\ (desired)\ output\ of\ output\ node\ k$

$\frac{dP}{do}$ - derivative of the performance function with respect to the actual output of the network

$\frac{dO_k}{dp_r}$ - derivative of the activation function at output node k, with respect to its weighted inputs

Softmax and Cross Entropy Pairing

Cross-Entropy Penalty Function

$$\rho_c = \sum_{n=1}^{N} \sum_{k=1}^{q} t_{nk} \log\left(\frac{t_{nk}}{z^*_{nk}}\right).$$

Actual output of output node k, for the given pattern n

Derivative

$$\frac{\partial \rho_c}{\partial z_k} = \left(z^*_k - t_k\right)$$

Reference: On the Pairing of the Softmax Activation and Cross-Entropy Penalty Functions and the Derivation of the Softmax Activation Function by R.A. Dunne & N.A. Campbell

$$\frac{\partial \rho_c}{\partial W_{jk}} = \left(z^*_k - t_k\right) * \frac{\partial z_k}{\partial W_{jk}}$$

Z_k is the net input

Weight-update

$$W'_{jk} = W_{jk} - \eta * \left(z^*_k - t_k\right) * \frac{\partial z_k}{\partial W_{jk}}$$

Actual output of output node k, given pattern n

Softmax and Cross Entropy Pairing

Cross-Entropy Penalty Function

$$\rho_c = \sum_{n=1}^{N} \sum_{k=1}^{q} t_{nk} \log\left(\frac{t_{nk}}{z^*_{nk}}\right).$$

http://neuralnetworksanddeeplearning.com/chap3.html#the_cross-entropy_cost_function

Weight-update

$$W'_{jk} = W_{jk} - \eta * \left(z^*_k - t_k\right) * O_j$$

Actual output of output node k, given pattern n

Target Value

- During training it is impossible for the outputs to reach **0** or **1** (with finite weights)
- Customary to use **0.1** and **0.9** as targets
- But, most **termination criteria**,

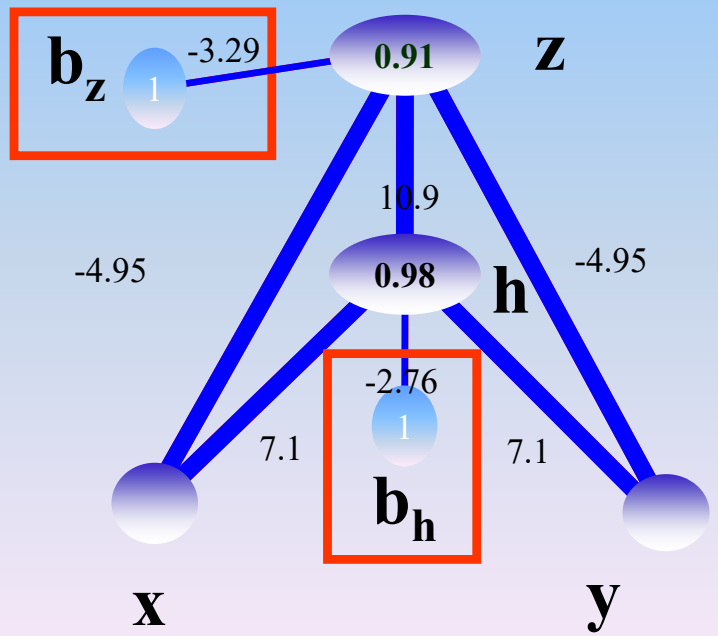
  e.g. **small change in training** or **validation error** will stop training before targets are reached.

Bias Units

Multi-Layer Feed-Forward Neural Network

Why do we need BIAS UNITS?

Apart from improving the speed of learning for some problems (XOR problem), bias units or threshold nodes are required for universal approximation. Without them, the feedforward network always assigns 0 output to 0 input. **Without thresholds, it would be impossible to approximate functions which assign nonzero output to zero input.** Threshold nodes are needed in much the same way that the constant polynomial '1' is required for approximation by polynomials.



Using Different Activation Functions



Using Different Activation Functions

Now that we understand the composition of the weight update formulas at the different layers of the network, it is now easy for us to change the activation function used in the network, as well as the required weight update formula.

If we opt to change the **activation function** used by a node in the network, the **derivative of the activation function** in the weight update formula needs to be changed as well.



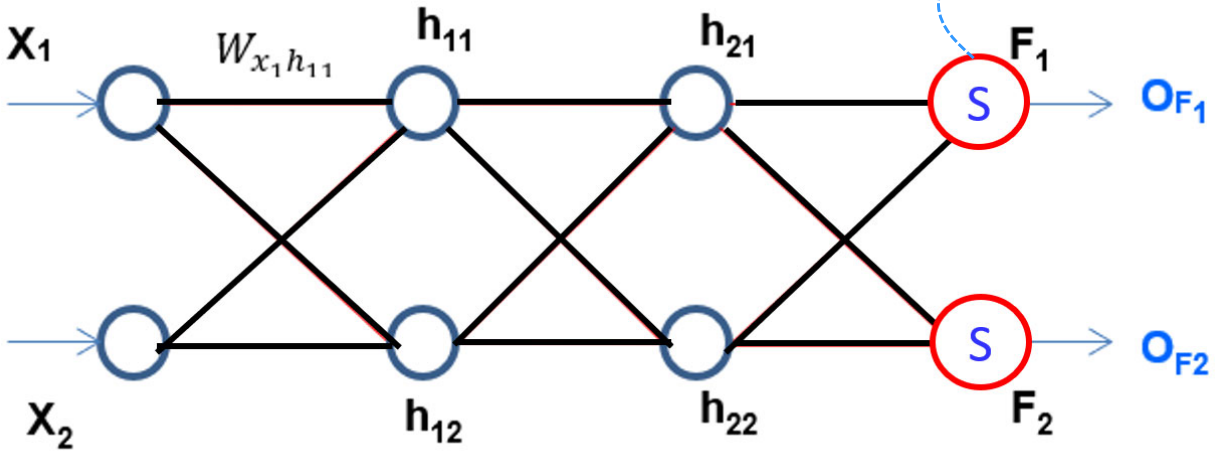
Using the Sigmoid Activation Function



Gradient Ascent

Output nodes using the sigmoid function as activation function.

Uses the Sigmoid Activation function



Error delta at the output node(s)

$$\delta_k = \overbrace{(t_k - O_k)}^{\frac{dP}{do}} \overbrace{Ok(1 - O_k)}^{\frac{dO_k}{dp_r}}$$

Derivative of the Sigmoid function with respect to its input

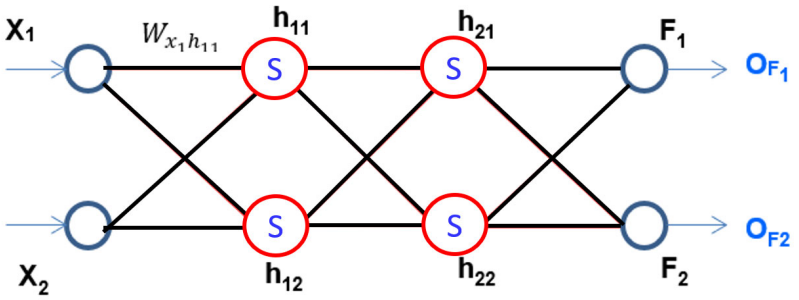
$O_k - activation\ function\ of\ output\ node\ k$
$p_r - weighted\ input\ of\ output\ node\ k$
$t_k - target\ (desired)\ output\ of\ output\ node\ k$

$\frac{dP}{do}$ - derivative of the performance function with respect to the actual output of the network

$\frac{dO_k}{dp_r}$ - derivative of the activation function at output node k, with respect to its weighted inputs

Gradient Ascent

Hidden nodes using the sigmoid function as activation function.



S - uses the Sigmoid Activation function

Error delta at a hidden node:

error signals back-propagated to node j

Weights of the links carrying the *back-propagated error signals*

$$\delta_j = \frac{dO_j}{dp_r} \sum_k \delta_k W_{jk}$$

Derivative of the Sigmoid function with respect to its input

η – *learning rate*
k – *output unit*
J – *hidden node that is directly connected at node k (which is residing at a higher layer)*
δ_k – *error delta at node k*
O_j – *output of unit j*

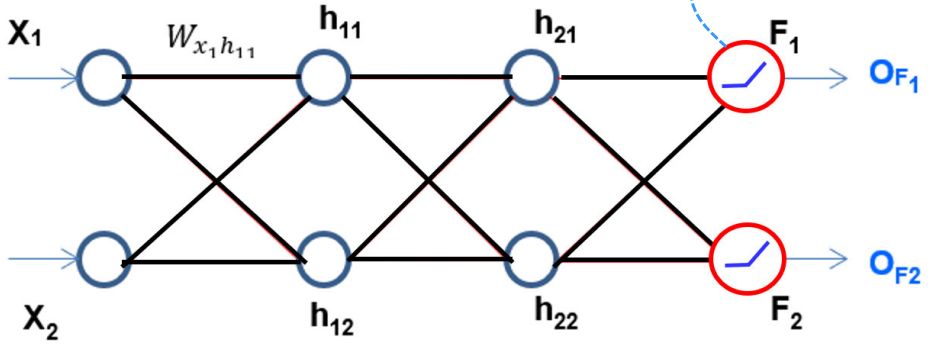
Using the ReLU Activation Function



Gradient Ascent

Uses the **ReLU** Activation function

Output nodes using the ReLU function as activation function.



Error delta at the output node(s)

- If input to node is greater than zero

Derivative of the **ReLU** function with respect to its input

$$\delta_k = \underbrace{(t_k - O_k)}_{\frac{dP}{do}} \underbrace{(\mathbf{1})}_{\frac{dO_k}{dp_r}}$$

- Else

$$\delta_k = \mathbf{0}$$

$O_k - activation\ function\ of\ output\ node\ k$
$p_r - weighted\ input\ of\ output\ node\ k$
$t_k - target\ (desired)\ output\ of\ output\ node\ k$

$\frac{dP}{do}$ - derivative of the performance function with respect to the actual output of the network

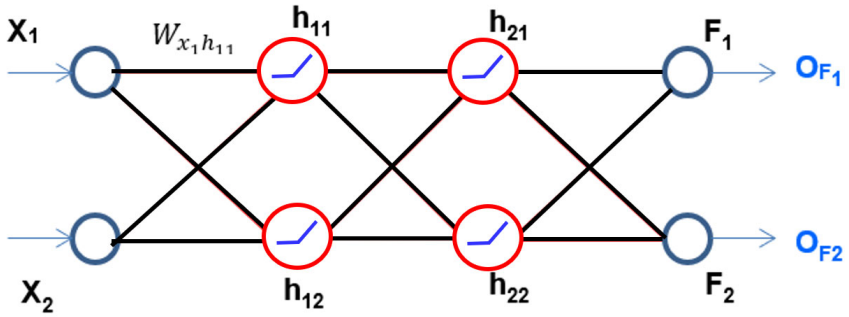
$\frac{dO_k}{dp_r}$ - derivative of the activation function at output node k, with respect to its weighted inputs

Gradient Ascent

Hidden nodes using the ReLU function as activation function.



- uses the **ReLU** Activation function



Error delta at a hidden node:

- If input to node is greater than zero

Derivative of the **ReLU** function with respect to its input

error signals back-propagated to node j

Weights of the links carrying the *back-propagated error signals*

$$\delta_j = (\mathbf{1}) \sum_k \delta_k W_{jk}$$

- Else

$$\delta_j = \mathbf{0}$$

η – *learning rate*
k – *output unit*
J – *hidden node that is directly connected at node k (which is residing at a higher layer)*
δ_k – *error delta at node k*
O_j – *output of unit j*



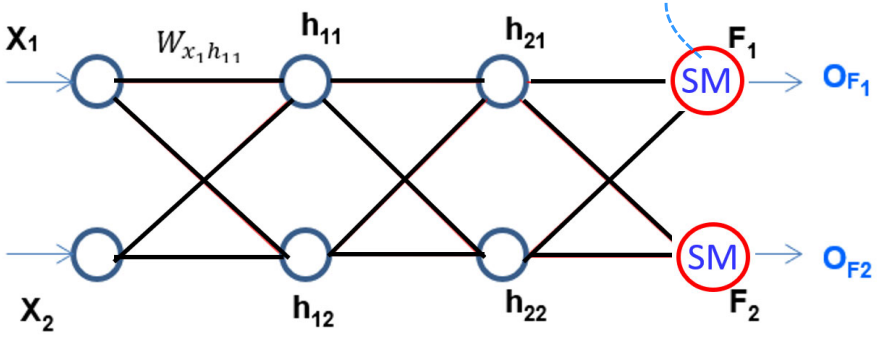
Using the Softmax Activation Function at the Output Layer



Gradient Ascent

Uses the **Softmax** Activation function

Output nodes using the Softmax function as activation function.



Error delta at the output node(s)

- The error delta is computed based on the derivative of the **cross-entropy penalty function** and the **derivative of the softmax function**. Interestingly, using the pairing of the two, the **error delta** is significantly simplified.

$$\delta_k = (t_k - O_k)$$

$O_k - activation\ function\ of\ output\ node\ k$
$t_k - target\ (desired)\ output\ of\ output\ node\ k$

Derivative of Activation Functions

https://github.com/tensorflow/playground/blob/master/nn.ts

```
/** Built-in activation functions */
export class Activations {
  public static TANH: ActivationFunction = {
    output: x => (<any>Math).tanh(x),
    der: x => {
      let output = Activations.TANH.output(x);
      return 1 - output * output;
    }
  };
  public static RELU: ActivationFunction = {
    output: x => Math.max(0, x),
    der: x => x <= 0 ? 0 : 1
  };
  public static SIGMOID: ActivationFunction = {
    output: x => 1 / (1 + Math.exp(-x)),
    der: x => {
      let output = Activations.SIGMOID.output(x);
      return output * (1 - output);
    }
  };
  public static LINEAR: ActivationFunction = {
    output: x => x,
    der: x => 1
  };
}
```



Gradient Descent

Performance Function

P = (1/2) * ((desired_output – actual_output)^2)

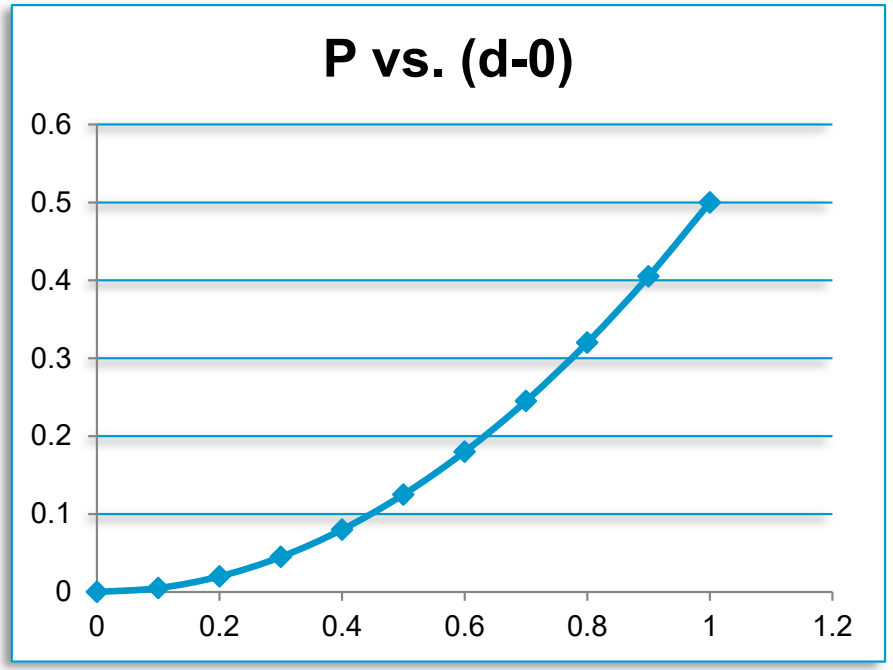
desired output	actual output	difference	Performance
1	1	0	0
1	0.9	0.1	0.005
1	0.8	0.2	0.02
1	0.7	0.3	0.045
1	0.6	0.4	0.08
1	0.5	0.5	0.125
1	0.4	0.6	0.18
1	0.3	0.7	0.245
1	0.2	0.8	0.32
1	0.1	0.9	0.405
1	0	1	0.5

actual output - desired output

$$\frac{dP}{do} = (o - d)$$

$$\Delta w \ \alpha \ \frac{\partial \mathrm{P}}{\partial w}$$

$$w = w - \eta \frac{\partial \mathrm{P}}{\partial w}$$



When training the network, we would like to minimise P

Gradient Descent

Summary of Weight Update Formulas

$$\frac{dP}{do} = (o - d)$$

$$\Delta w \ \alpha \ \frac{\partial P}{\partial w}$$

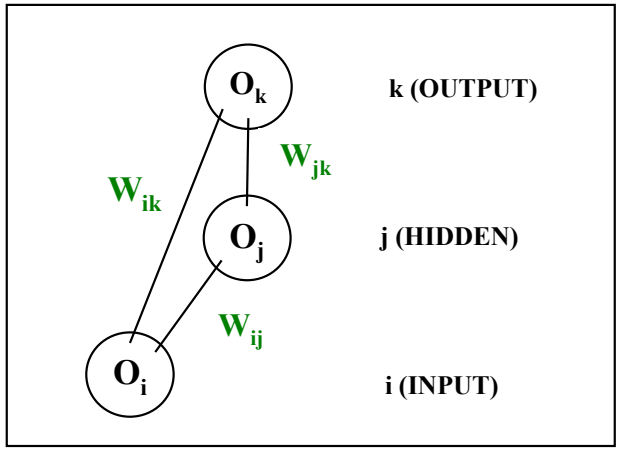
$$w = w - \eta \frac{\partial P}{\partial w}$$

To adjust weights leading into an output unit, W_{jk}:

$$\delta_k = (O_k - t_k) Ok(1 - O_k)$$
$$W_{jk} = W_{jk} - \eta \delta_k O_j$$

$$W_{ik} = W_{ik} - \eta \delta_k O_i$$

η – learning_rate
δ_k – error_signal
O_j – output_of_unit_j



To adjust weights leading into a hidden unit, W_{ij}:

$$\delta_j = Oj(1 - O_j) \sum_k \delta_k W_{jk}$$

$$W_{ij} = W_{ij} - \eta \delta_j O_i$$

η – learning_rate
δ_j – error_signal at unit j
O_i – output_of_unit_i

Hyperparameter

- A **hyperparameter** is any number used in the neural network which cannot be learned by the network. An example would be the **learning rate**, **number of neurons in the hidden layer** or **number of hidden layers, momentum rate**.

- This means that learning cannot adjust hyperparameters; They have to be adjusted manually by training the network on training set (typically 80% of data set) and testing it on the **validation set**. We do this process of tweaking hyperparameters, training and then testing until we get the desired classification accuracy.

Shuffling the Examples

- Shuffle the training set so that successive examples never (or rarely) belong to the same class.

Example:

Syntax: Class, 16 input features

G,4,9,5,6,6,8,5,4,3,7,5,11,6,8,5,11
X,5,11,8,8,4,8,8,1,8,10,5,7,3,8,4,8
X,3,9,5,7,4,8,7,3,8,5,6,8,2,8,6,7
C,2,3,3,1,1,5,9,4,6,11,9,11,1,9,2,8
C,7,10,5,5,2,6,8,6,8,11,7,11,2,8,5,9
E,3,7,4,5,4,7,7,5,8,8,8,9,3,9,6,9
E,6,9,4,4,2,7,7,4,7,10,6,10,1,9,7,9
E,3,4,3,6,2,3,8,6,10,7,6,15,0,8,7,8
...
and so on.

This data set needs some shuffling!

Measure of Performance

Sum of Squared Error

- This error is minimised during training.

$$E = \frac{1}{2}\sum_{i=1}^{n}\sum_{k=1}^{m}(T_{ik} - O_{ik})^2$$

where
T_{ik} is the target output of node k for input pattern **i**;
O_{ik} is the actual output of node k for input pattern **i**;
m is the total number of output units;
n is the number of patterns;
i is the input pattern

Mean squared error

$$MSE = \frac{\sum_{i=1}^{n}\sum_{j=1}^{m}(o_{ij} - t_{ij})^2}{n}$$

- **n** is the number of patterns
- **m** is the number of components in the output vector
- **o** is the output of a single neuron j
- **t** is the target for the single neuron j
- **i** denotes the input pattern

Root mean squared error

$$RMSE = \sqrt{\frac{\sum_{i=1}^{n}\sum_{j=1}^{m}(o_{ij} - t_{ij})^2}{n}}$$

- **n** is the number of patterns
- **m** is the number of components in the output vector
- **o** is the output of a single neuron j
- **t** is the target for the single neuron j
- **i** denotes the input pattern

- This measure corresponds to the **Euclidean norm or L_2 norm,** denoted by $\|\cdot\|_2$.
- RMSE is generally the preferred performance measure for regression tasks

Mean absolute error

$$MAE = \frac{\sum_{i=1}^{n}\sum_{j=1}^{m}|o_{ij} - t_{ij}|}{n}$$

- **n** is the number of patterns
- **m** is the number of components in the output vector
- **o** is the output of a single neuron j
- **t** is the target for the single neuron j
- **i** denotes the input pattern

- for regression tasks with many outliers, you may consider using the Mean Absolute Error
- This measure corresponds to the **Manhattan norm** or **L_1 norm**, denoted by $\|\cdot\|_1$.

Cross-Entropy Error

$$\rho_c = \sum_{n=1}^{N} \sum_{k=1}^{q} t_{nk} \log\left(\frac{t_{nk}}{z^*_{nk}}\right)$$

- **N** is the number of patterns
- **q** is the number of output neurons
- $\mathbf{Z^*_{nk}}$ is the actual output of neuron k, for pattern n
- $\mathbf{t_{nk}}$ is the target output for neuron k, using pattern n

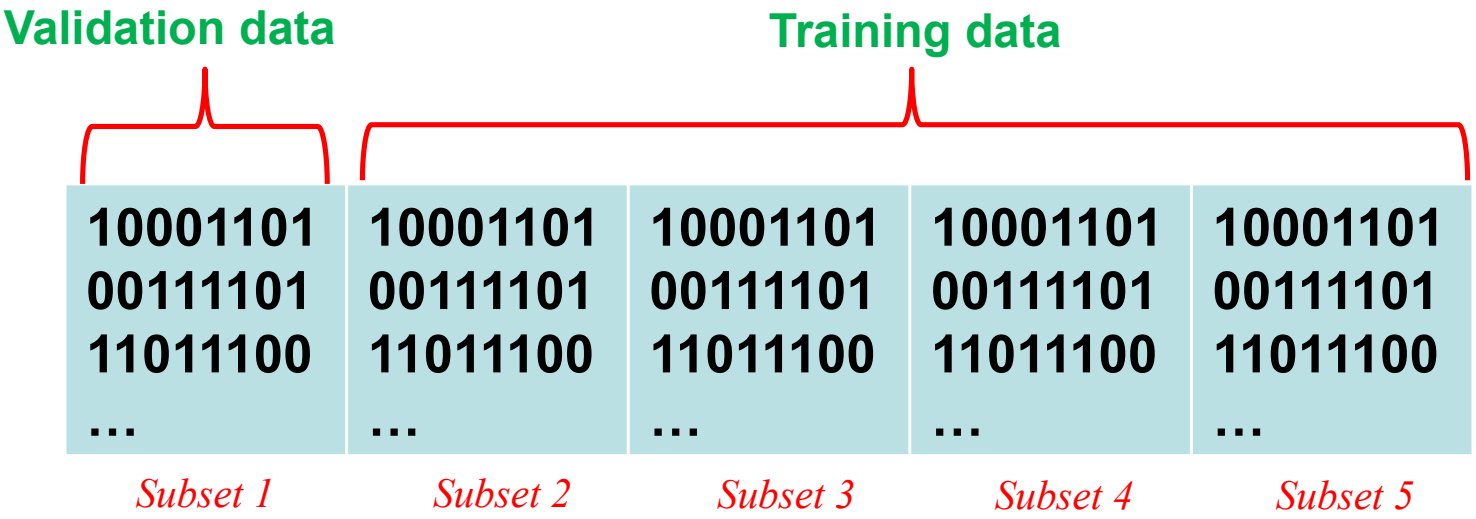
- Generally for multiple classification tasks

Reference: On the Pairing of the Softmax Activation and Cross-Entropy Penalty Functions and the Derivation of the Softmax Activation Function by R.A. Dunne & N.A. Campbell

Cross Validation

- Split data set (randomly) into **k equal size subsets**.

e.g. (k=5)



- Of the k subsets, a single subset is retained as the **validation data** for testing the model, and the remaining k − 1 subsets are used as **training data**.
- Repeat k times
- Average the results on validation data

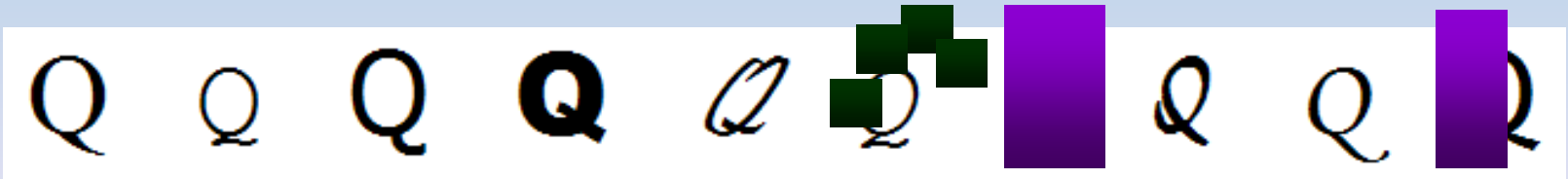
Use if data is scarce.

Training the Network

Other Tips for Training (from the Welstead book)

In a system to be used for a **real-world application**, such as character recognition, you would want the network to be able to handle **not only pixel noise**, but also **size variance** (slightly smaller letters), some **rotation**, **reflections**, as well as some **variations in font style**.

To produce such a robust network classifier, you would need to add representative samples to the training set. For example, samples of rotated versions of a character, reflected images, etc. could be added to the training set.



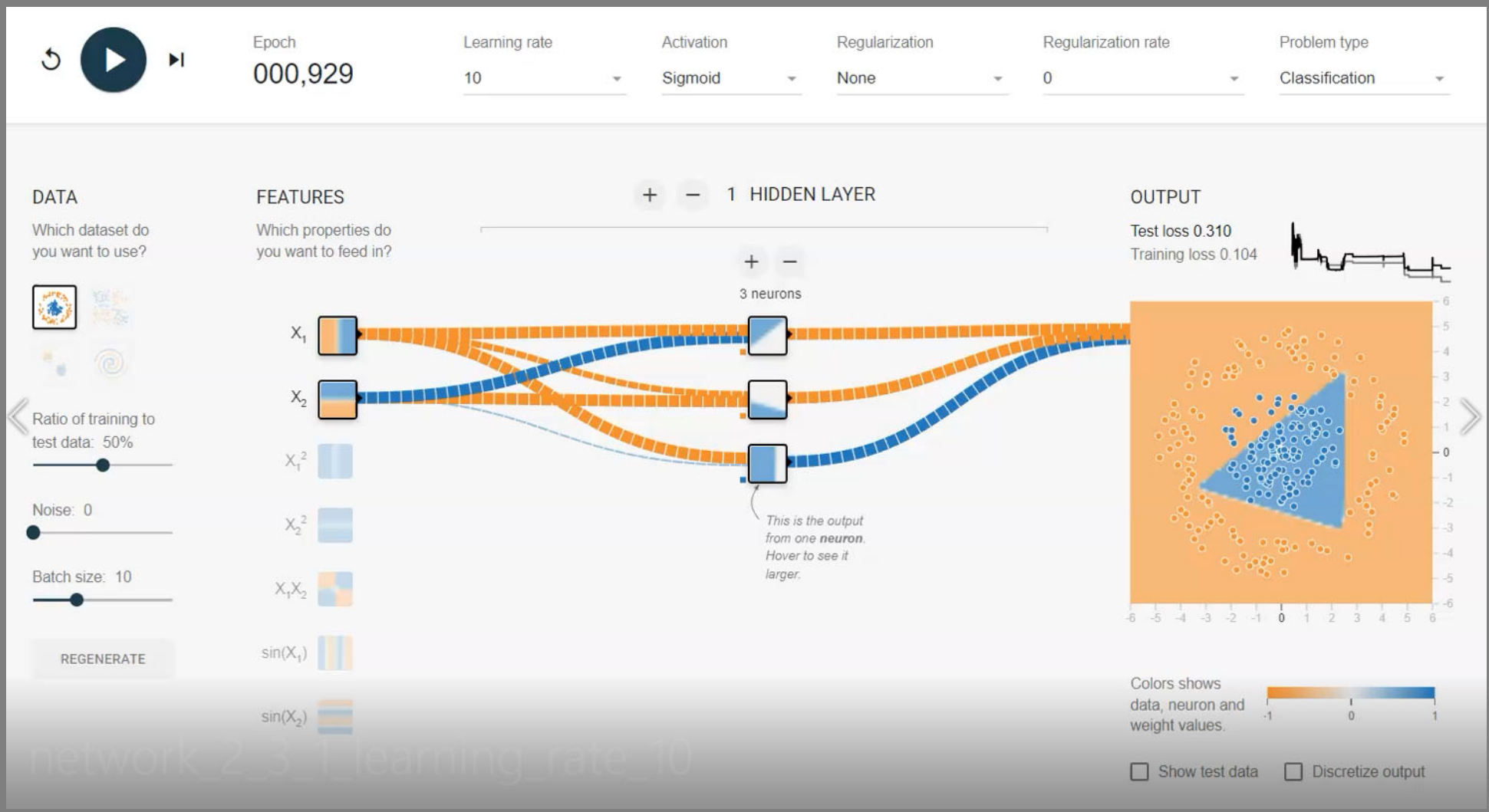
Experimenting with Neural Nets using Tensorflow playground

https://playground.tensorflow.org

Concentric Circles Data Set

Too high learning rate

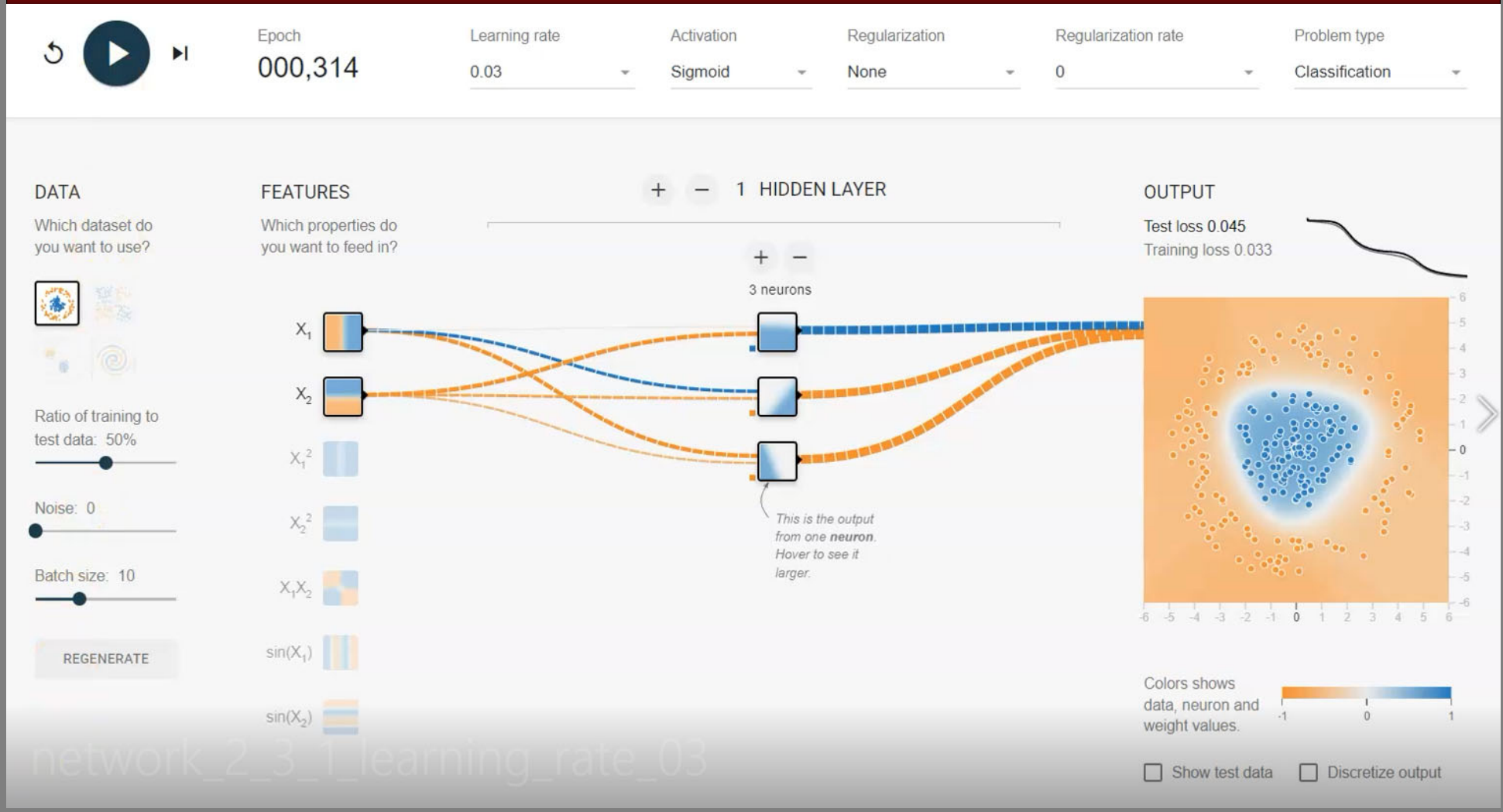
CIRCLES DATASET



network_2_3_1_learning_rate_10

Adjusted learning rate

CIRCLES DATASET

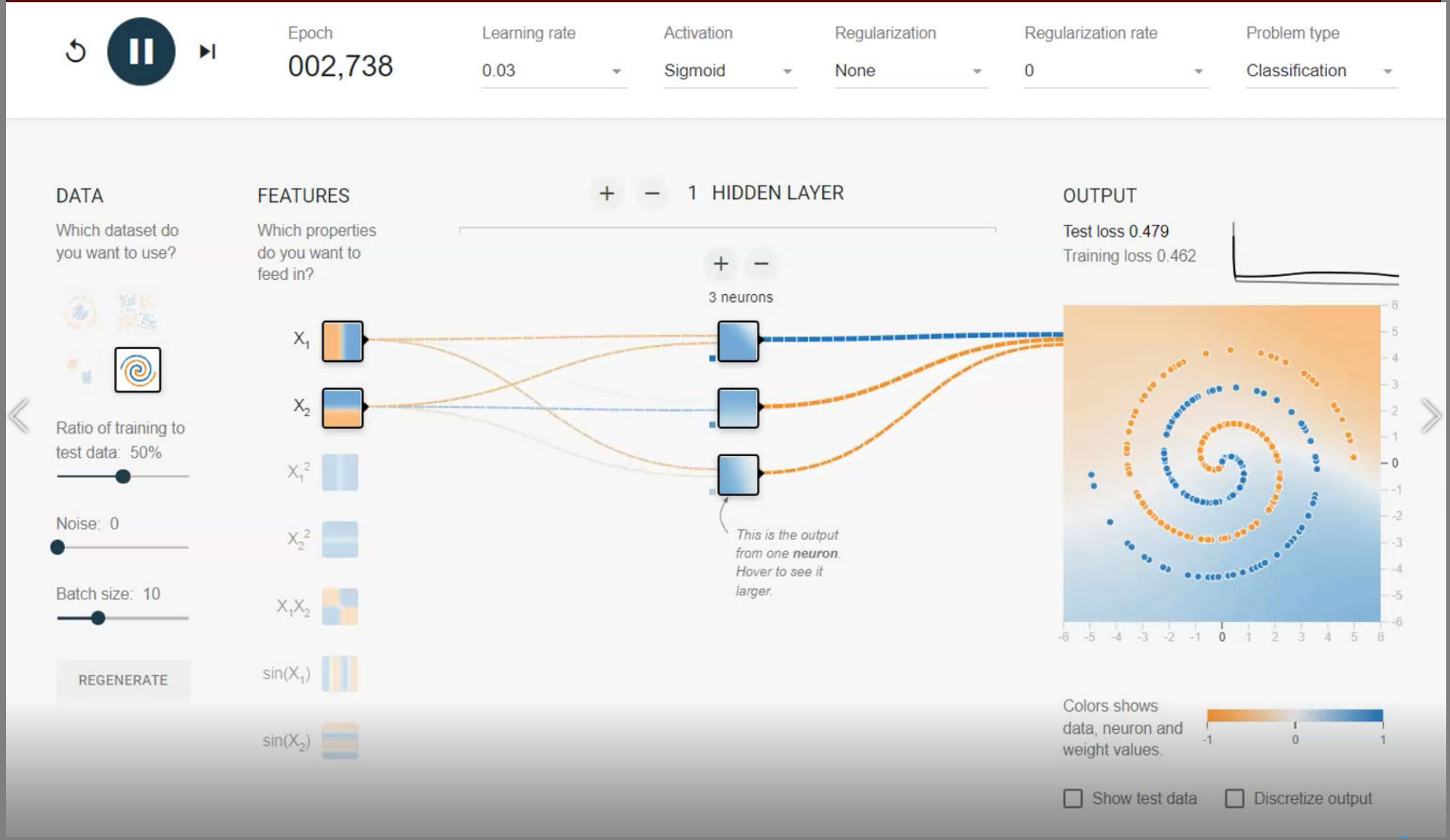


Spiral Data Set

A more challenging classification problem

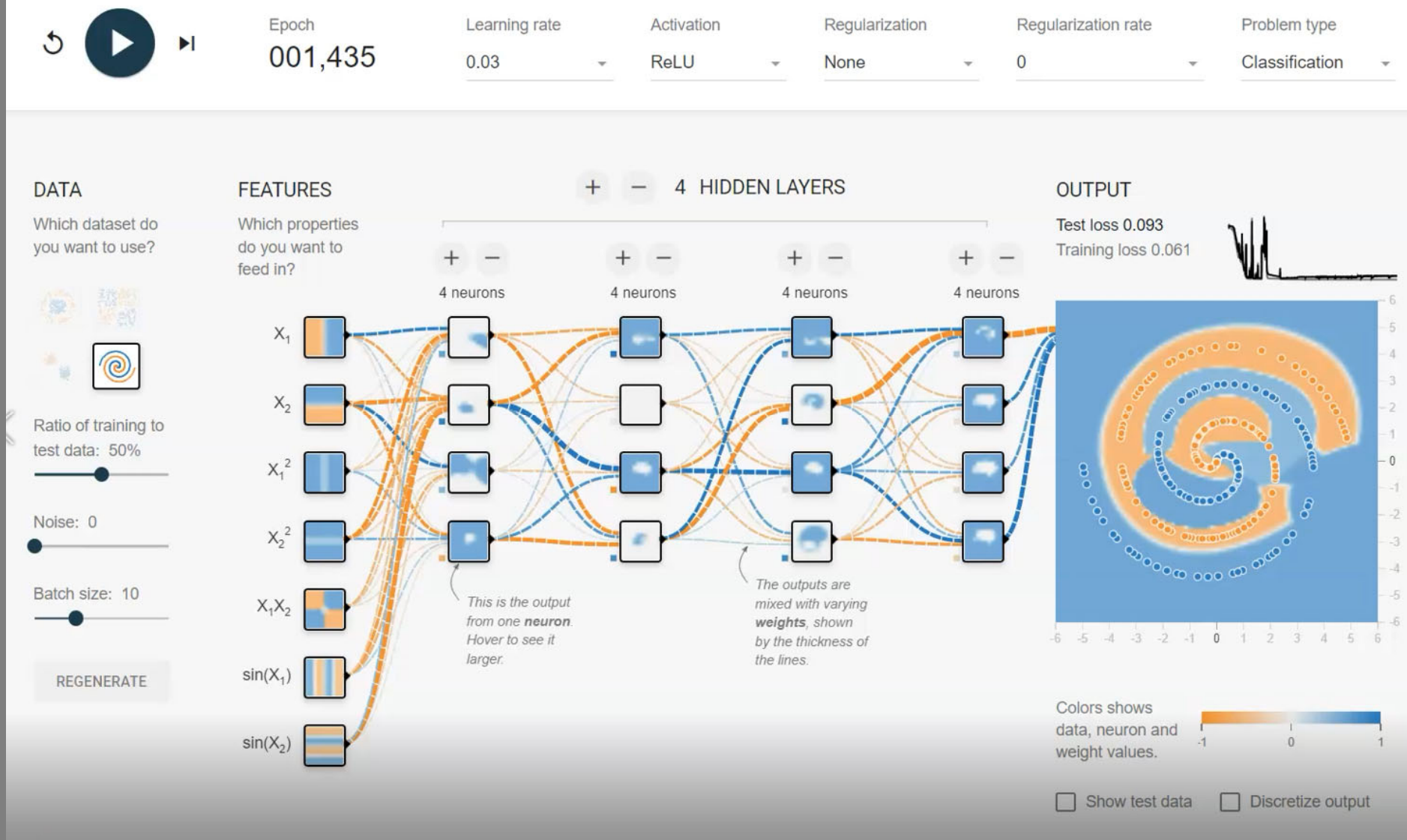
Failed attempt

SPIRAL DATASET



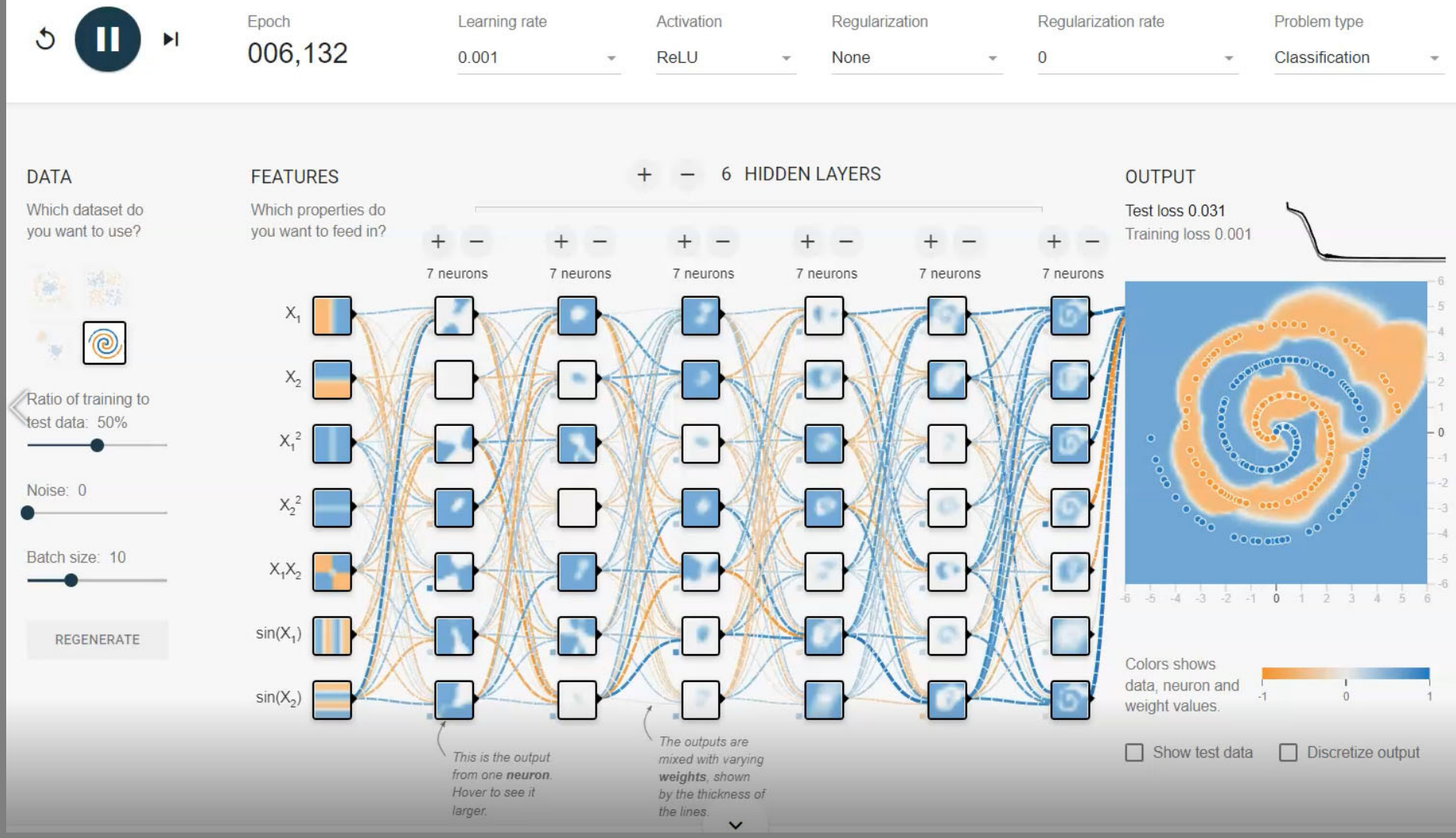
Feature Engineering

SPIRAL DATASET



Feature Engineering + Deep Network

SPIRAL DATASET



Deep Learning

SPIRAL DATASET

Epoch 001,078 | Learning rate 0.03 | Activation ReLU | Regularization None | Regularization rate 0 | Problem type Classification

DATA

Which dataset do you want to use?

Ratio of training to test data: 50%

Noise: 0

Batch size: 10

REGENERATE

FEATURES

Which properties do you want to feed in?

X_1
X_2
X_1^2
X_2^2
X_1X_2
$\sin(X_1)$
$\sin(X_2)$

6 HIDDEN LAYERS

7 neurons | 7 neurons | 7 neurons | 7 neurons | 7 neurons | 7 neurons

*This is the output from one **neuron**. Hover to see it larger.*

*The outputs are mixed with varying **weights**, shown by the thickness of the lines.*

OUTPUT

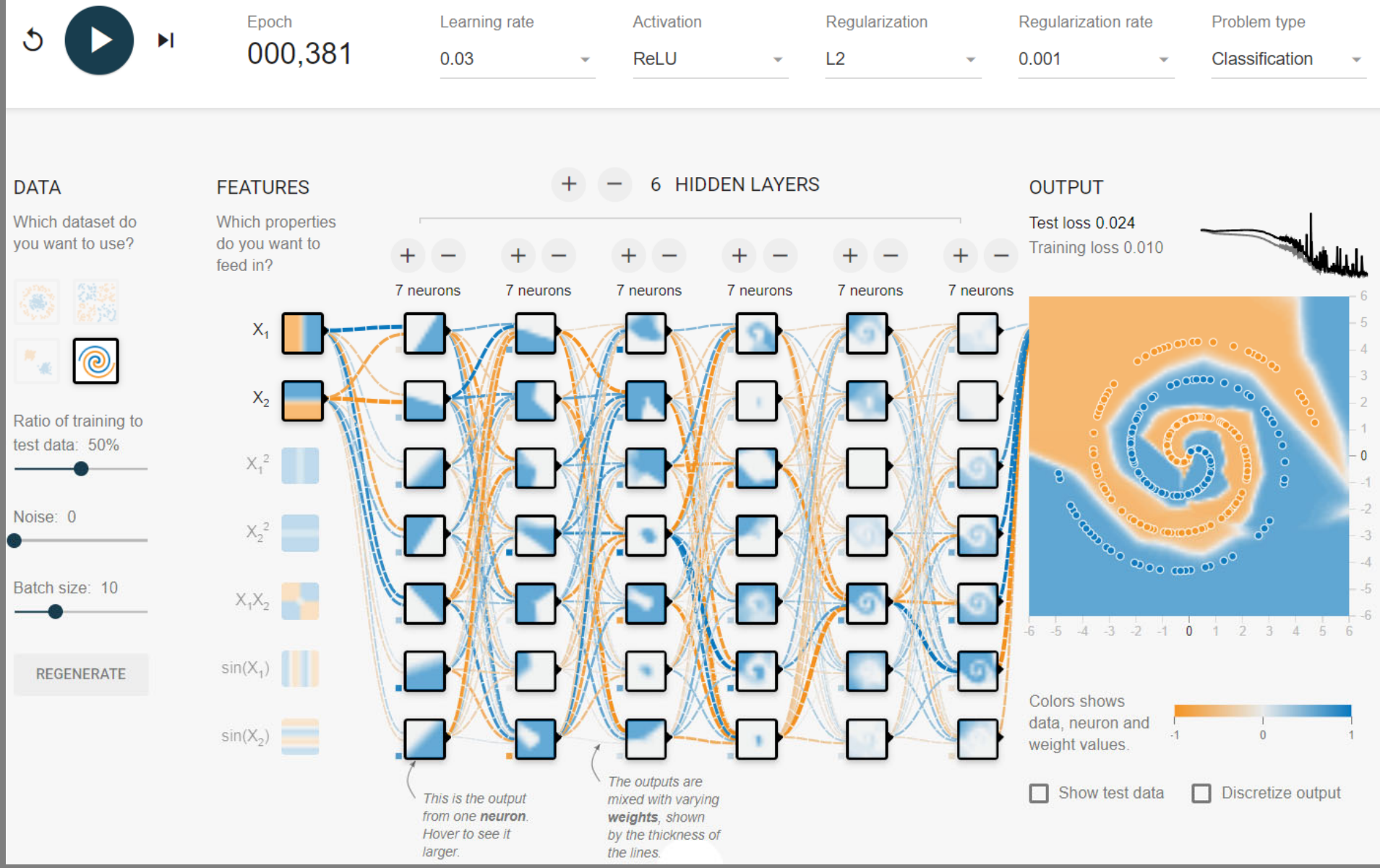
Test loss 0.029
Training loss 0.005

Colors shows data, neuron and weight values. -1 0 1

Show test data | Discretize output

Deep Learning with Regularization

SPIRAL DATASET



~The End.~

Tensor Flow Codes

https://github.com/tensorflow/playground/blob/master/nn.ts

```
/** Built-in activation functions */
export class Activations {
  public static TANH: ActivationFunction = {
    output: x => (<any>Math).tanh(x),
    der: x => {
      let output = Activations.TANH.output(x);
      return 1 - output * output;
    }
  };
  public static RELU: ActivationFunction = {
    output: x => Math.max(0, x),
    der: x => x <= 0 ? 0 : 1
  };
  public static SIGMOID: ActivationFunction = {
    output: x => 1 / (1 + Math.exp(-x)),
    der: x => {
      let output = Activations.SIGMOID.output(x);
      return output * (1 - output);
    }
  };
  public static LINEAR: ActivationFunction = {
    output: x => x,
    der: x => 1
  };
}
```

References

1. MIT OpenCourseWare
2. www.cim.mcgill.ca/~jer/courses/ai/readings/backprop.pdf
3. Neural Network and Fuzzy Logic Applications in C/C++ (Wiley Professional Computing) by Stephen Welstead
4. Introduction to Deep Learning by Sandro Skansi
5. Hands-On Machine Learning with Scikit-Learn and Tensorflow: Concepts, Tools, and Techniques to Build Intelligent Systems
6. http://www.cse.unsw.edu.au/~billw/cs9444/crossentropy.html
7. On the Pairing of the Softmax Activation and Cross-Entropy Penalty Functions and the Derivation of the Softmax Activation Function by R.A. Dunne & N.A. Campbell

